

DVWA vulnerability Assessment Report

Submitted by: Mohammed Ifaz M

Date: 20 August 2025

DVWA - Lab Notes

This document contains step-by-step notes, screenshots, and explanations for practicing Command Injection, SQL injection, Reflected XSS on DVWA (Damn Vulnerable Web Application). Each test includes input, screenshots, explanation, impact, and fix.

DVWA Setup (Docker in Kali)

1. Pulled DVWA image:

```
docker pull vulnerables/web-dvwa
```

2. Started container:

```
docker run --rm -it -p 8080:80 vulnerables/web-dvwa
```

3. Accessed DVWA at:

```
http://127.0.0.1:8080
```

4. Default credentials:

- Username: admin
- Password: password

DVWA Command Injection

Test 1 — Normal Input

Low Security

****Input:****
127.0.0.1

****Screenshot (Before):****



Vulnerability: Command Injection

Ping a device

Enter an IP address:

****Screenshot (After):****



Vulnerability: Command Injection

Ping a device

Enter an IP address:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.095 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.321 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.184 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.117 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.095/0.179/0.321/0.088 ms
```

****Explanation:****

The app executes a simple ping. No injection yet, but confirms it takes user input and executes system-level commands.

****Impact:****

If unprotected, this function could allow an attacker to run arbitrary commands.

****Fix:****

Whitelist IP addresses and avoid direct shell execution.

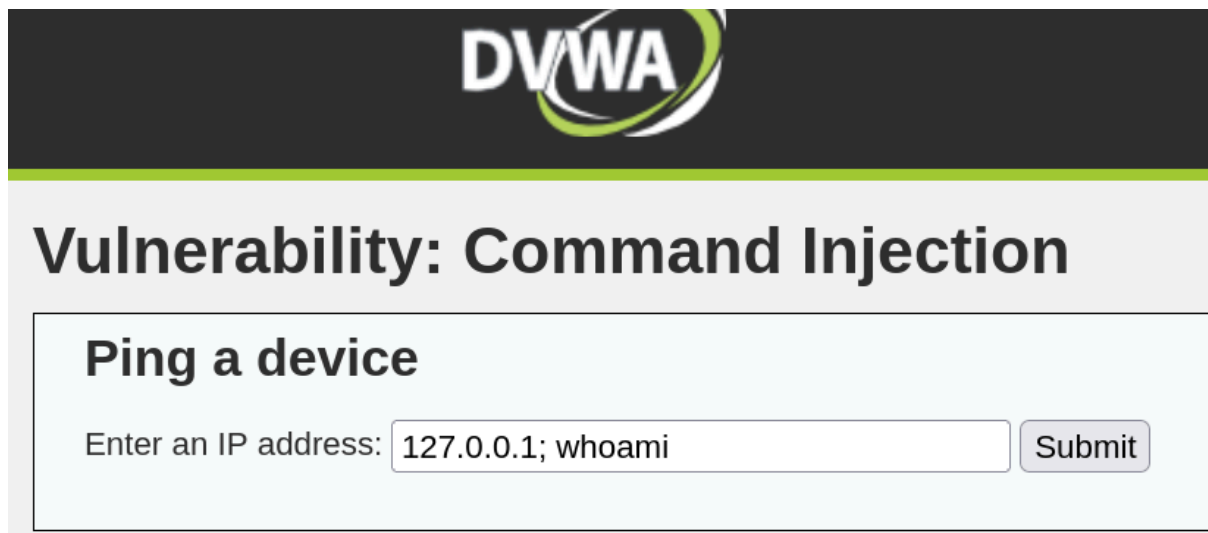
Test 2 — Injecting a System Command

Low Security

****Input:****

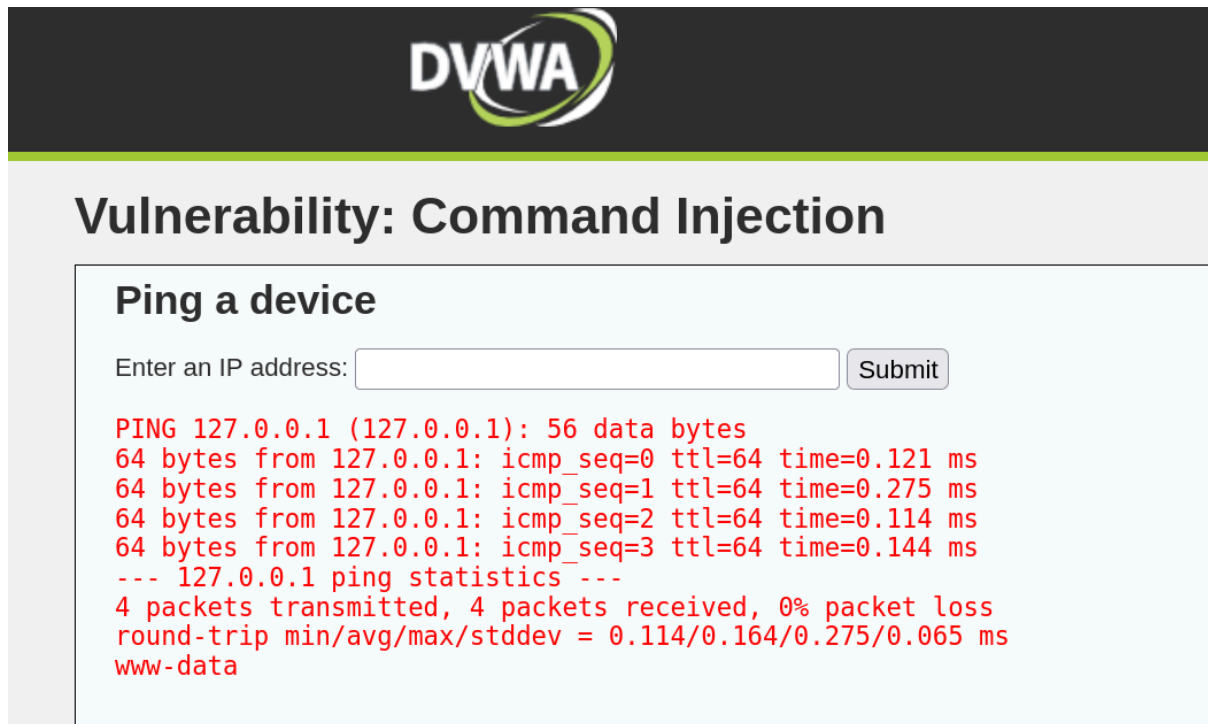
127.0.0.1; whoami

****Screenshot (Before):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is a dark header with the DVWA logo. Below the header is a light gray section titled "Vulnerability: Command Injection". Underneath this is a white box titled "Ping a device". Inside this box, there is a label "Enter an IP address:" followed by a text input field containing the value "127.0.0.1; whoami". To the right of the input field is a "Submit" button.

****Screenshot (After):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the page title is "Vulnerability: Command Injection". The main section is titled "Ping a device". It contains a form with the label "Enter an IP address:" followed by a text input field and a "Submit" button. Below the form, the output of a successful ping command is displayed in red text:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.121 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.275 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.114 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.144 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.114/0.164/0.275/0.065 ms
www-data
```

****Explanation:****

The ';' lets you chain another command. Here, we learn which user runs the webserver.

****Impact:****

We now have arbitrary command execution on the server.

****Fix:****

Block special characters (;, &, |, &&). Use parameterized functions instead of shell calls.

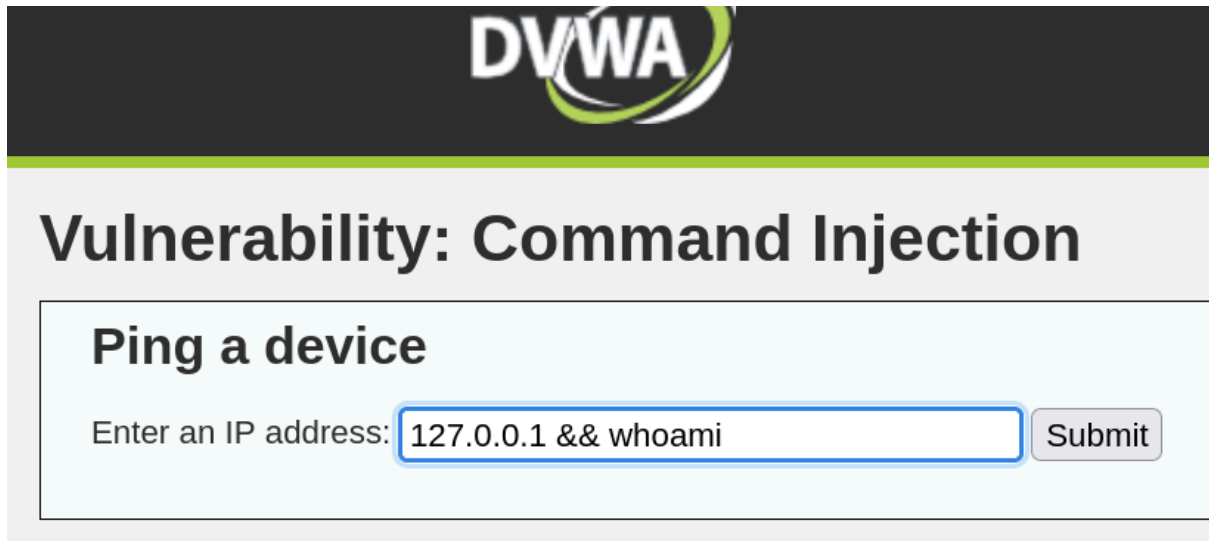
Test 3 — Injecting a System Command

Medium Security

****Input:****

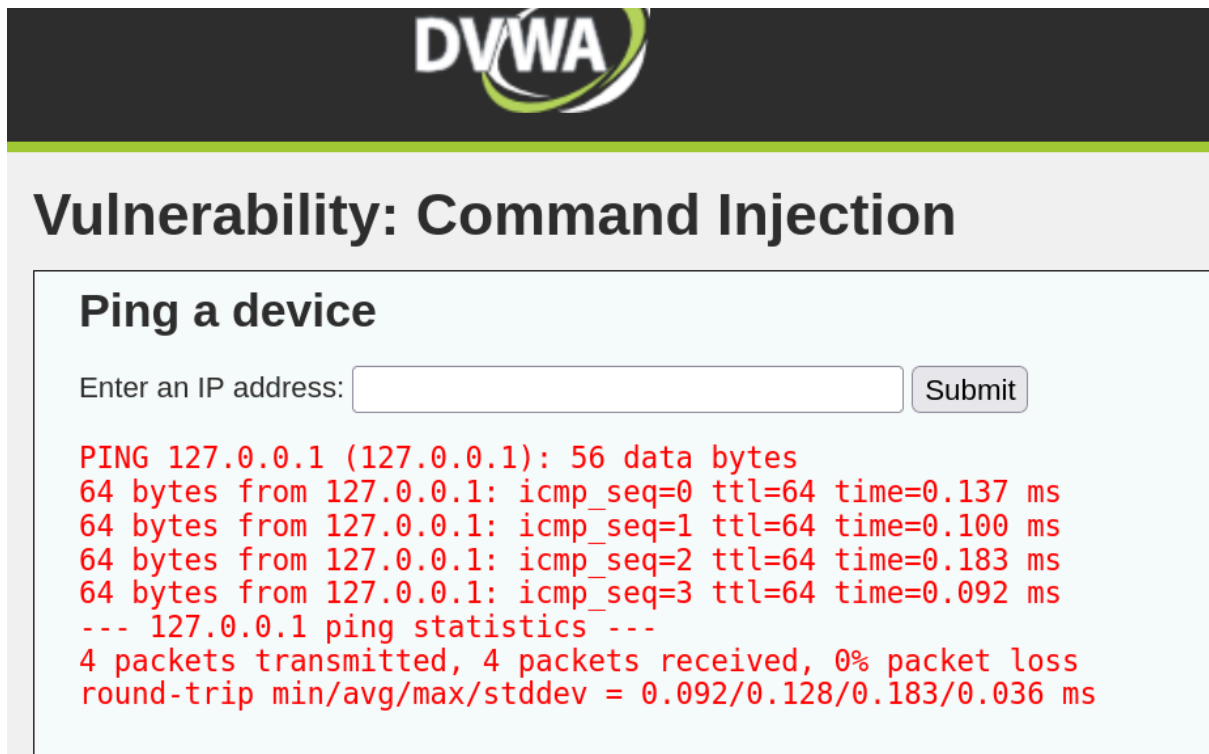
127.0.0.1 && whoami

****Screenshot (Before):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: Command Injection" is displayed. Underneath, there is a section titled "Ping a device". Below this title, there is a text input field labeled "Enter an IP address:" containing the text "127.0.0.1 && whoami". To the right of the input field is a "Submit" button.

****Screenshot (After):****



The screenshot shows the DVWA interface after the command injection. The title "Vulnerability: Command Injection" remains. Under the "Ping a device" section, the "Enter an IP address:" input field is now empty. Below the input field, the output of the command injection is displayed in red text:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.137 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.100 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.183 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.092 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.092/0.128/0.183/0.036 ms
```

****Explanation:****

Medium security blocks ; and &&, but | still works to chain commands.

****Impact:****

We again gain arbitrary command execution despite the filter.

****Fix:****

Block all shell metacharacters, not just a few. Use parameterized system calls.

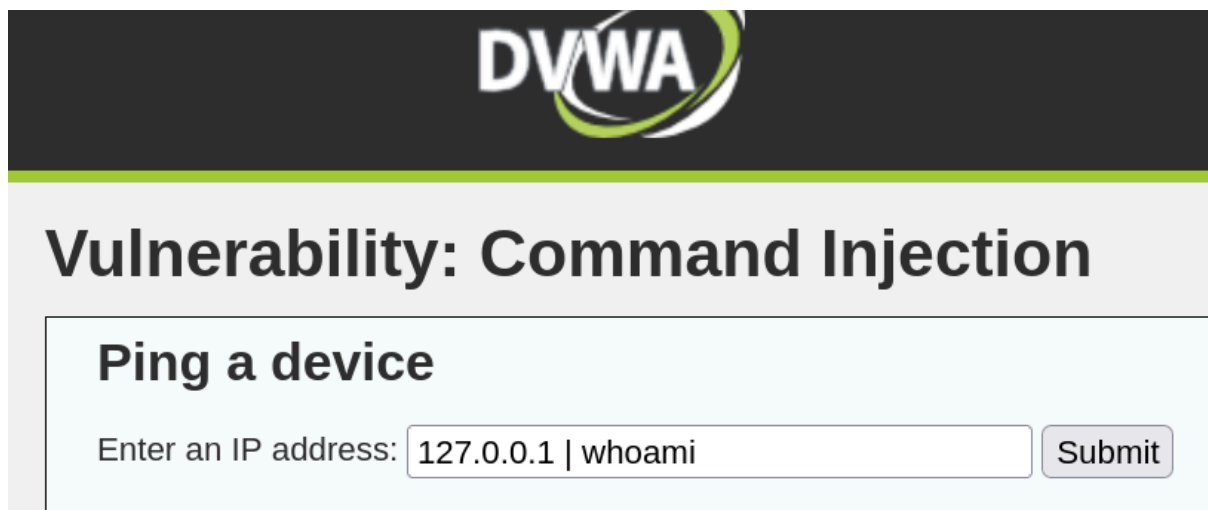
Test 4 — Injecting a System Command

High Security

****Input:****

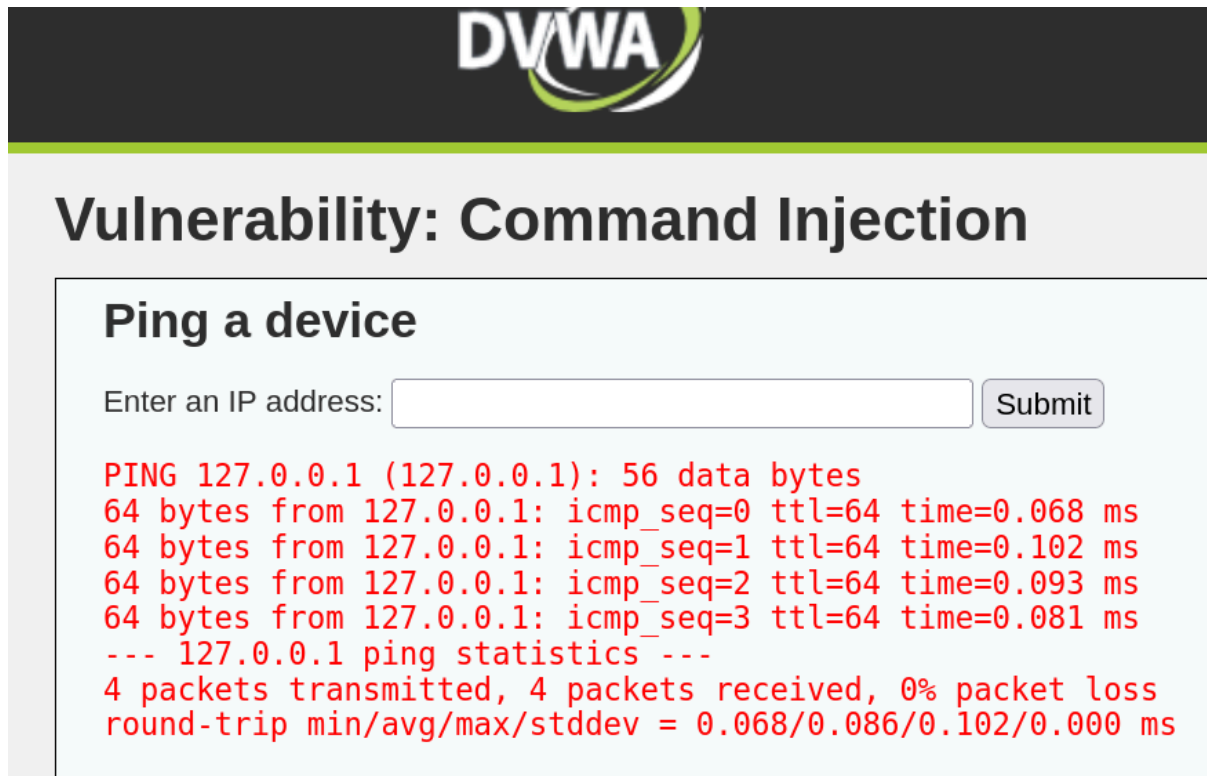
127.0.0.1 | whoami

****Screenshot (Before):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: Command Injection" is displayed. Underneath, there is a section titled "Ping a device". In this section, there is a label "Enter an IP address:" followed by a text input field containing the value "127.0.0.1 | whoami". To the right of the input field is a "Submit" button.

****Screenshot (After):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: Command Injection" is displayed. The main section is titled "Ping a device". It contains a form with the label "Enter an IP address:" followed by a text input field and a "Submit" button. Below the form, the output of a successful ping command is shown in red text:

```
PING 127.0.0.1 (127.0.0.1): 56 data bytes
64 bytes from 127.0.0.1: icmp_seq=0 ttl=64 time=0.068 ms
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.102 ms
64 bytes from 127.0.0.1: icmp_seq=2 ttl=64 time=0.093 ms
64 bytes from 127.0.0.1: icmp_seq=3 ttl=64 time=0.081 ms
--- 127.0.0.1 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max/stddev = 0.068/0.086/0.102/0.000 ms
```

****Explanation:****

In High mode, DVWA sanitizes dangerous characters like `;`, `|`, `&&`, `$()`, and backticks. The injected `whoami` command was ignored, so only the normal ping ran.

****Impact:****

No arbitrary code execution was possible. This shows how stricter input validation greatly reduces the risk of command injection.

****Fix:****

Validate input with strict whitelisting, avoid passing it to the shell, and use safe system or parameterized functions instead.

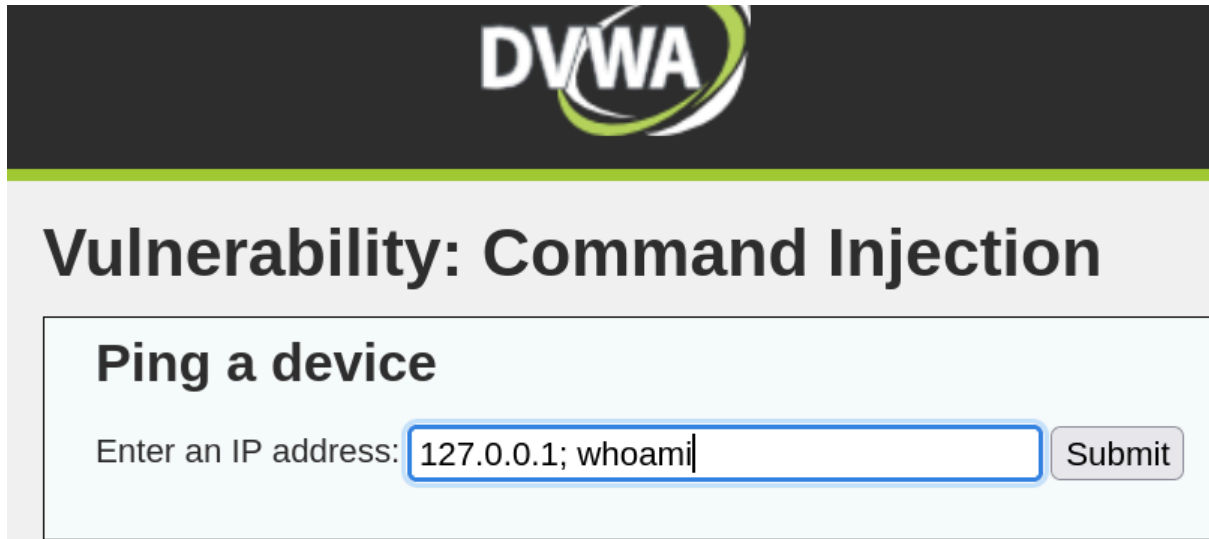
Test 5 — Injecting a System Command

Impossible Level (Secure)

****Input:****

127.0.0.1; whoami

****Screenshot (Before):****

A screenshot of the DVWA (Damn Vulnerable Web Application) interface. The top header is dark grey with the DVWA logo in white and green. Below the header, the title "Vulnerability: Command Injection" is displayed in a large, bold, black font. Underneath the title, there is a section titled "Ping a device" in bold black font. Below this section title, the text "Enter an IP address:" is followed by a text input field containing the value "127.0.0.1; whoami". To the right of the input field is a "Submit" button.

DVWA

Vulnerability: Command Injection

Ping a device

Enter an IP address:

****Screenshot (After):****

A screenshot of the DVWA interface after a failed command injection attempt. The layout is identical to the previous screenshot, but the input field is now empty. Below the input field, a red error message is displayed: "ERROR: You have entered an invalid IP." The "Submit" button remains visible to the right of the input field.

DVWA

Vulnerability: Command Injection

Ping a device

Enter an IP address:

ERROR: You have entered an invalid IP.

****Explanation:****

DVWA's Impossible level enforces strict whitelisting. Any input that isn't a valid IP address is rejected, which prevents command injection.

****Impact:****

Command injection is fully blocked. Attackers cannot chain or execute system commands.

****Fix:****

Already secure — the application properly validates inputs and avoids unsafe shell calls.

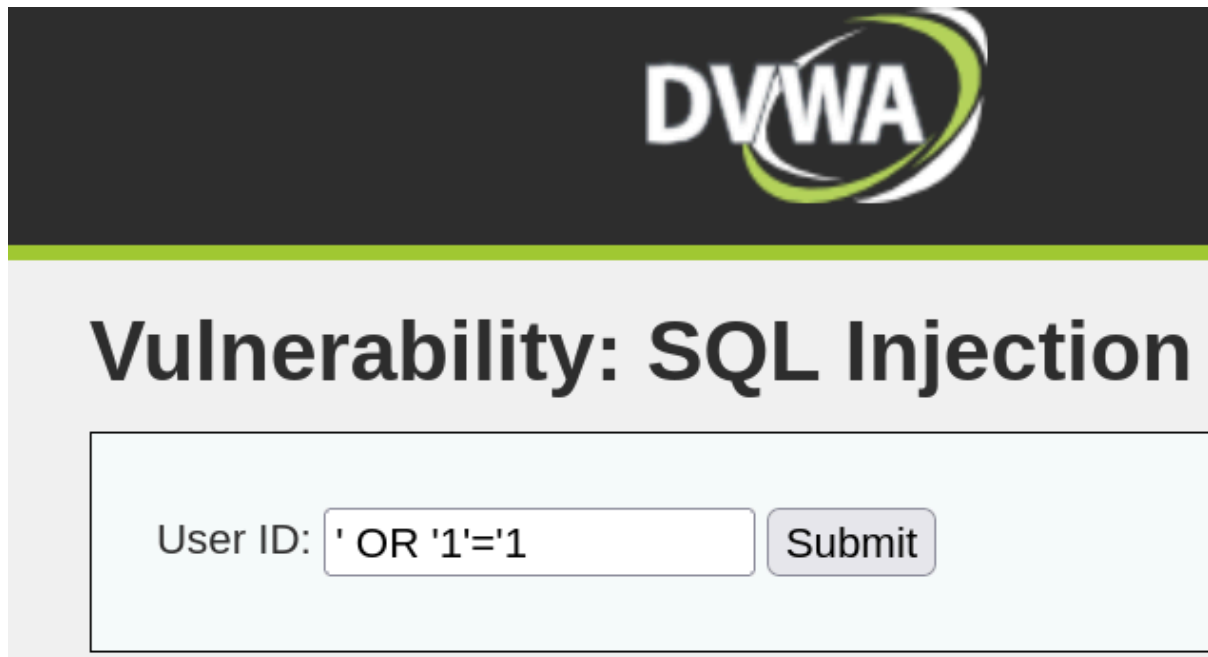
DVWA SQL Injection

Test 1 — Basic SQL Injection (Low)

****Input:****

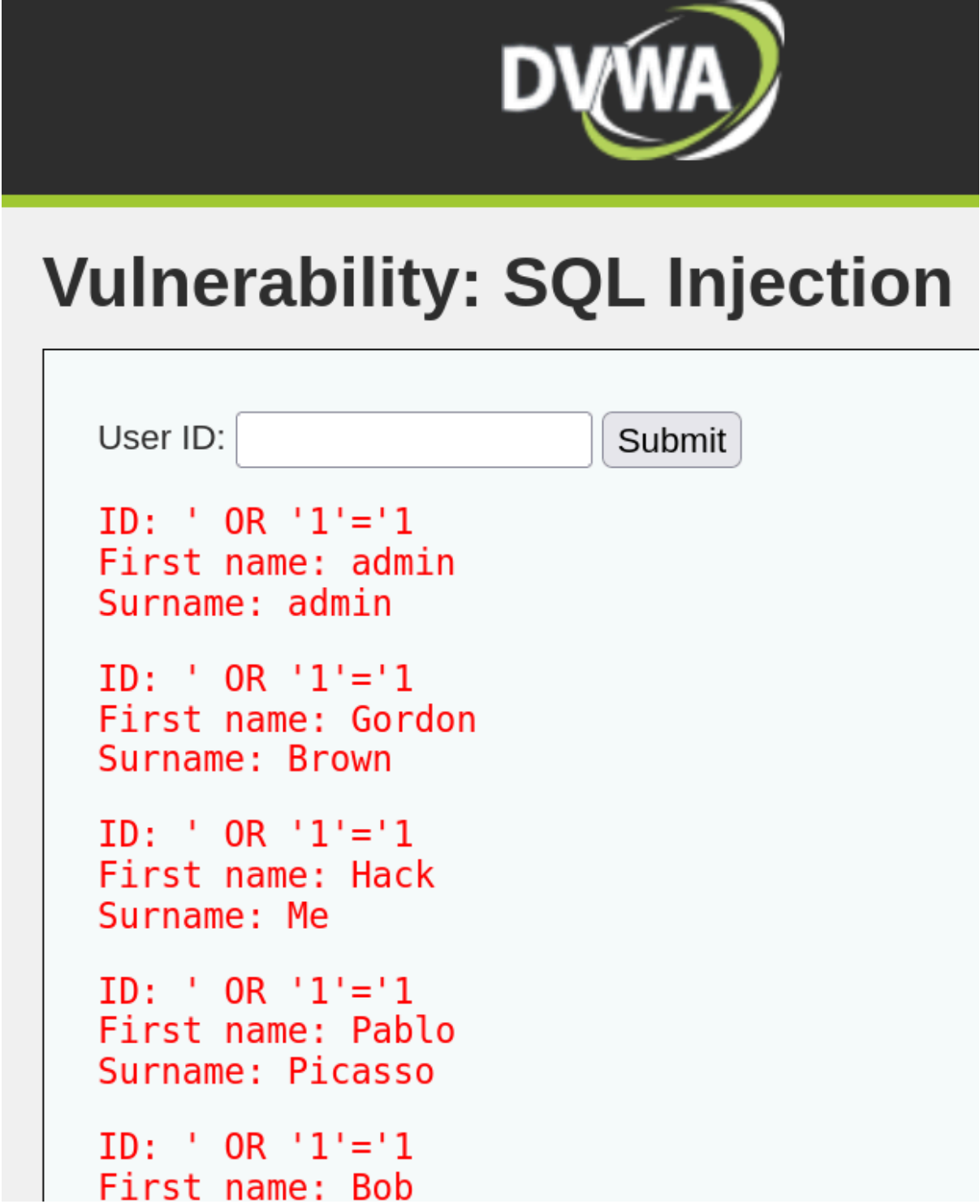
' OR '1'='1

****Screenshot (Before):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top, there is a dark header with the DVWA logo. Below the header, the title "Vulnerability: SQL Injection" is displayed in a large, bold font. Underneath the title, there is a light blue box containing a form. The form has a label "User ID:" followed by a text input field. The input field contains the text "' OR '1'='1". To the right of the input field is a "Submit" button.

****Screenshot (After):****



The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: SQL Injection" is displayed. The main content area shows a "User ID:" label followed by an empty text input field and a "Submit" button. Below the input field, the results of a SQL injection attack are displayed in red text. The results show five rows of user data, all retrieved due to the injected payload ' OR '1'='1'.

User ID:

ID: ' OR '1'='1
First name: admin
Surname: admin

ID: ' OR '1'='1
First name: Gordon
Surname: Brown

ID: ' OR '1'='1
First name: Hack
Surname: Me

ID: ' OR '1'='1
First name: Pablo
Surname: Picasso

ID: ' OR '1'='1
First name: Bob

****Explanation:****

' OR '1'='1 always evaluates to true, bypassing authentication or revealing all rows.

****Impact:****

Full access to database content or authentication bypass.

****Fix:****

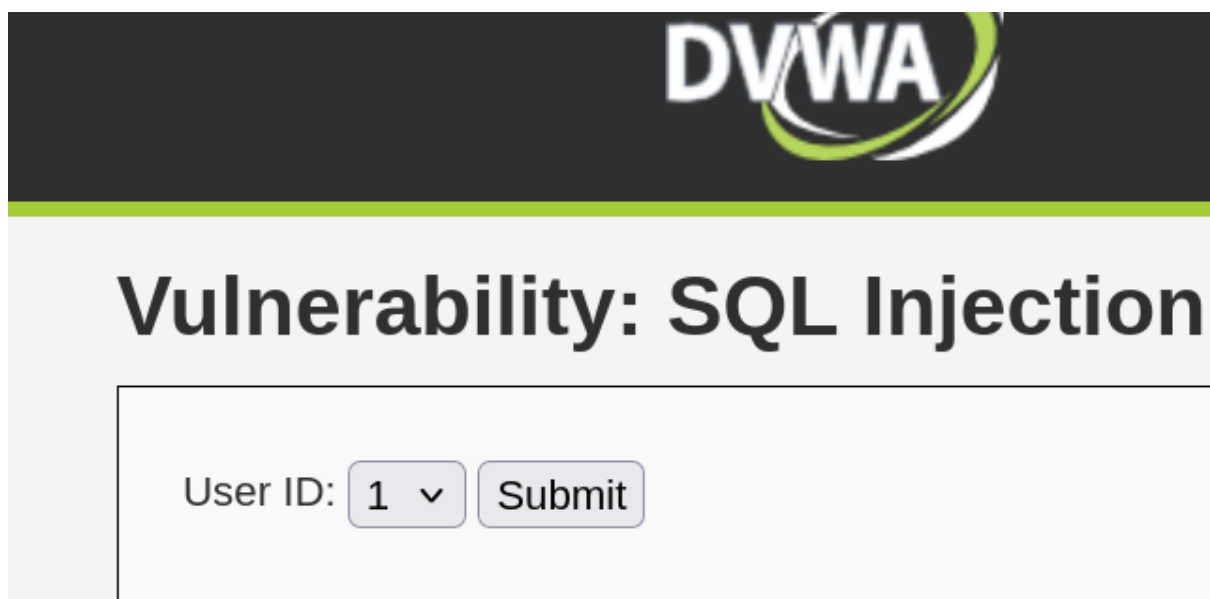
Use parameterized queries / prepared statements, input validation.

Test 2 — Medium Level

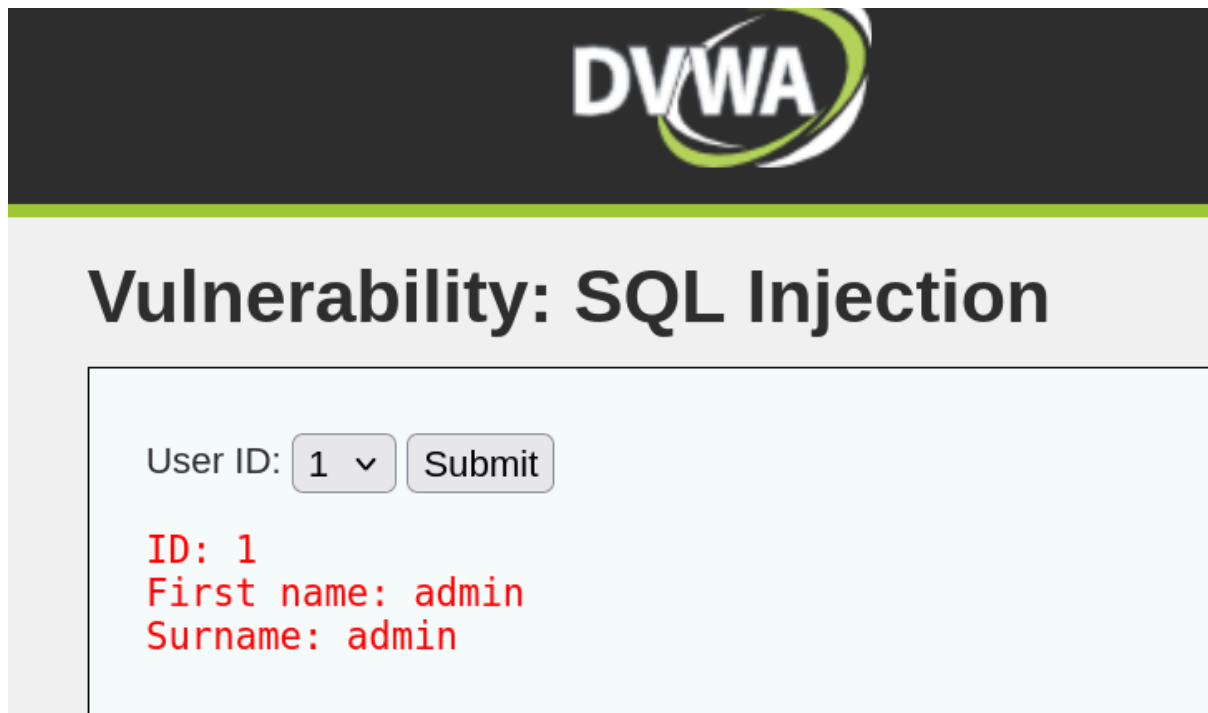
****Input:****

Selected User ID: 1

****Screenshot (Before):****



****Screenshot (After):****

The screenshot shows the DVWA (Damn Vulnerable Web Application) interface. At the top is the DVWA logo. Below it, the title "Vulnerability: SQL Injection" is displayed in a large, bold, black font. Underneath the title, there is a form with a label "User ID:" followed by a dropdown menu showing the value "1" and a "Submit" button. Below the form, the output is displayed in red text: "ID: 1", "First name: admin", and "Surname: admin".

DVWA

Vulnerability: SQL Injection

User ID:

ID: 1
First name: admin
Surname: admin

****Explanation:****

On Medium/High security levels, DVWA uses a dropdown for User IDs to prevent direct SQL Injection. You cannot type arbitrary payloads, but the vulnerability is still conceptually tested by observing how the selected ID affects output.

****Impact:****

SQL Injection is restricted due to input control, but in a real-world free-text scenario, the same database could be vulnerable if inputs were not validated.

****Fix:****

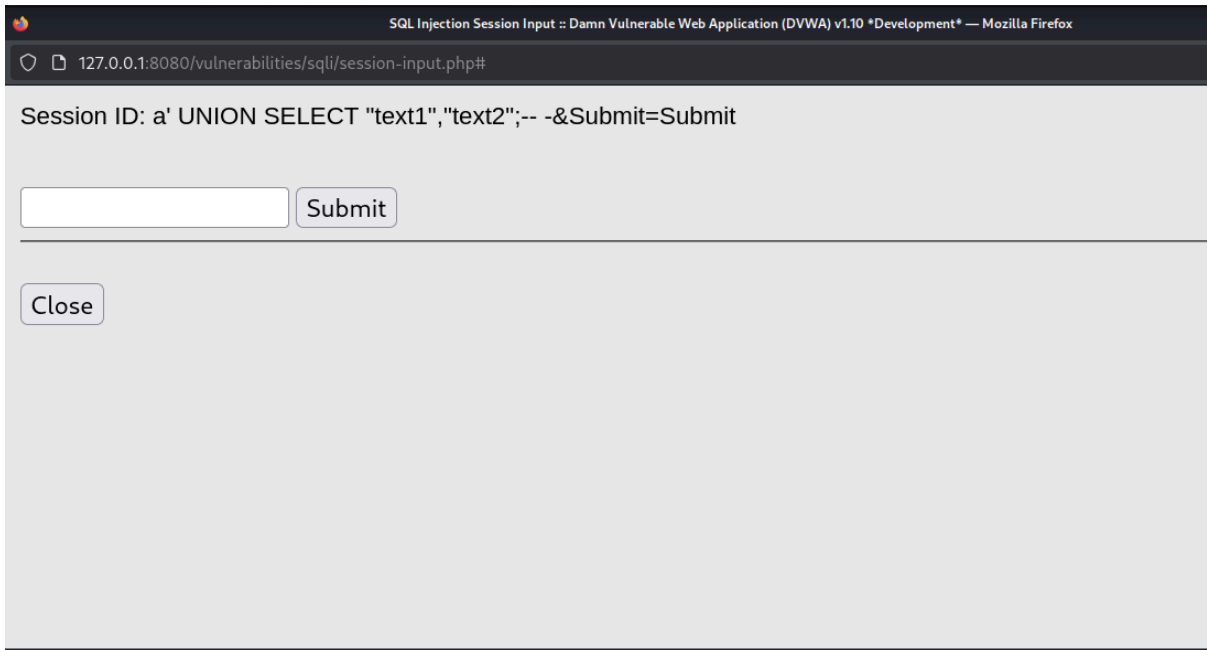
Use parameterized queries, proper input validation, and least privilege access.

Test 3 — High Security Level (Advanced / UNION SELECT)

****Input:****

ID: a' UNION SELECT "text1","text2";-- -&Submit=Submit

****Screenshot (Before):****

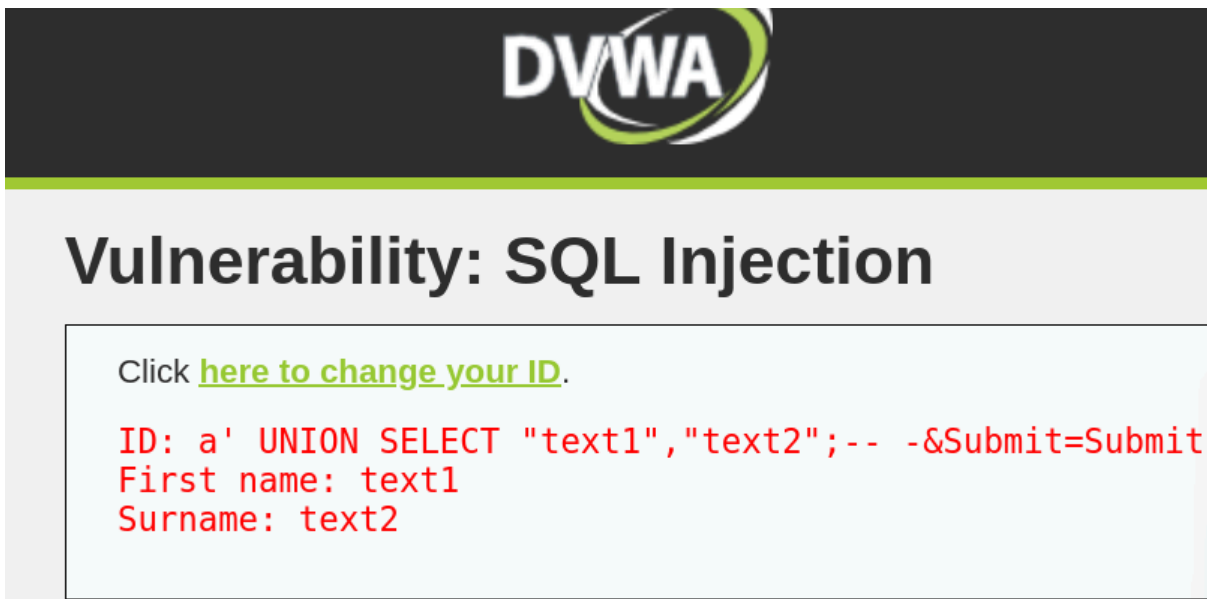


SQL Injection Session Input :: Damn Vulnerable Web Application (DVWA) v1.10 *Development* — Mozilla Firefox

127.0.0.1:8080/vulnerabilities/sqli/session-input.php#

Session ID: a' UNION SELECT "text1","text2";-- -&Submit=Submit

****Screenshot (After):****



DVWA

Vulnerability: SQL Injection

Click [here to change your ID.](#)

ID: a' UNION SELECT "text1","text2";-- -&Submit=Submit
First name: text1
Surname: text2

****Explanation:****

This UNION SELECT payload injects a custom row into the SQL query. Even at High security, it demonstrates how attackers could extract or inject data if input filtering or parameterization is weak.

****Impact:****

If input validation or parameterization is bypassed, an attacker could retrieve arbitrary data. This step shows how database exposure can occur without accessing real sensitive information.

****Fix:****

The vulnerability can be prevented by using parameterized queries, validating inputs strictly (allowing only numeric IDs), limiting database privileges, and avoiding detailed SQL error messages.

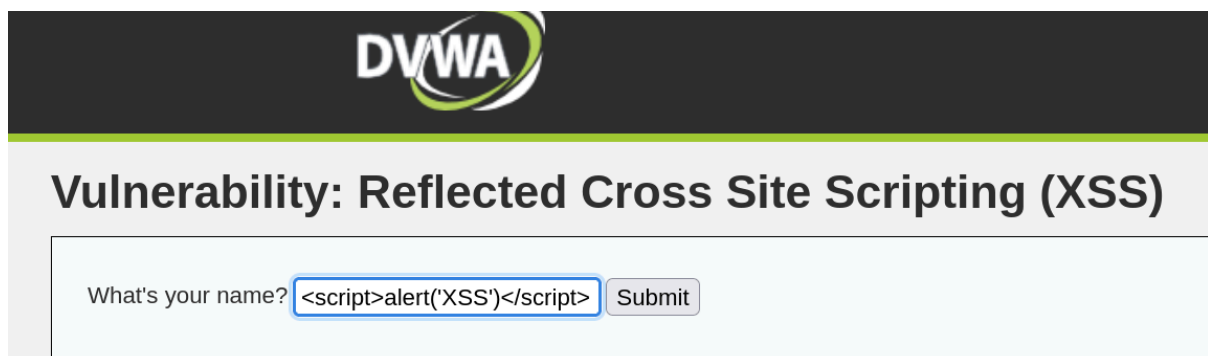
DVWA Reflected XSS

Test 1 — Low Security Level

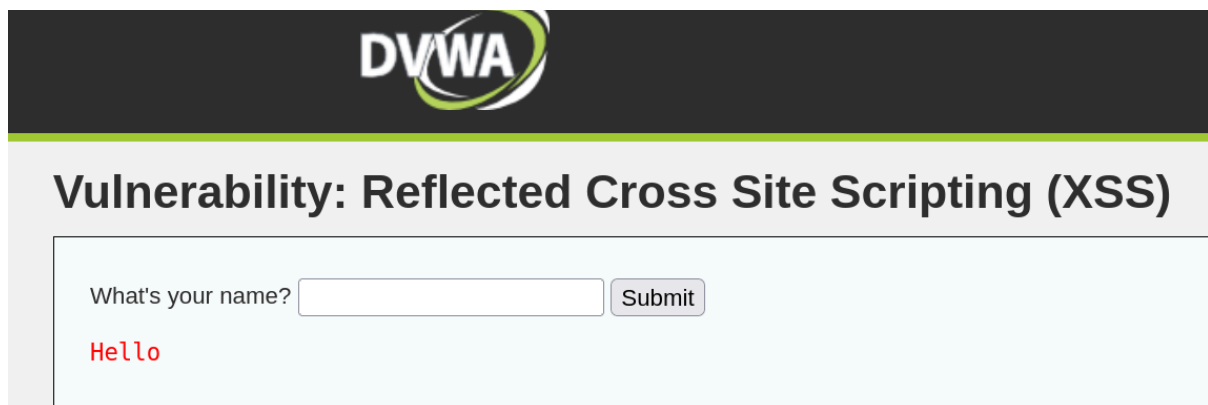
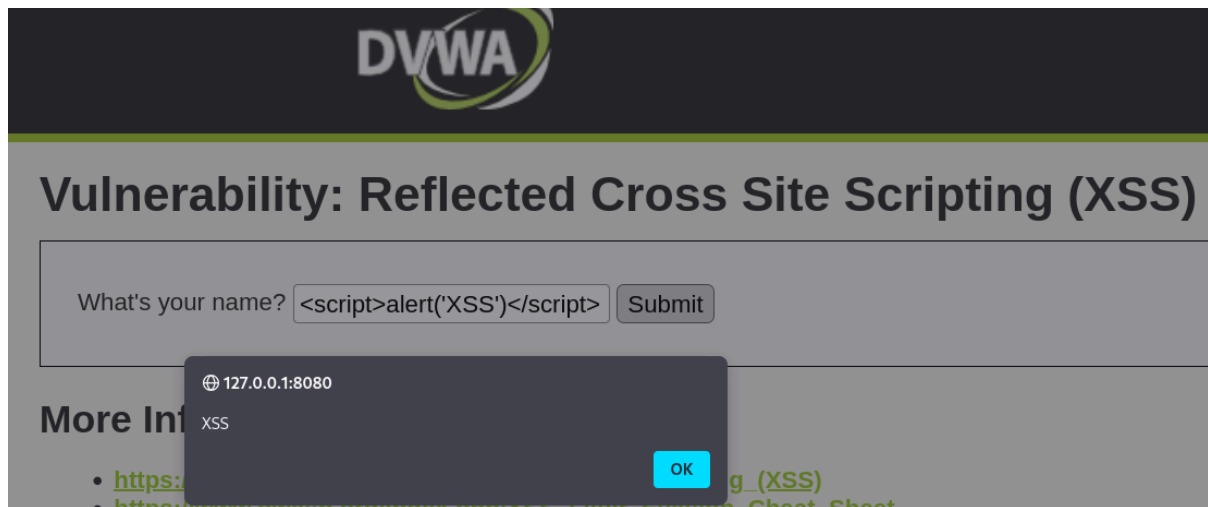
****Input:****

```
<script>alert('XSS')</script>
```

****Screenshot (Before):****



****Screenshot (After):****



****Explanation:****

Low security allows direct input of HTML/JavaScript code. The application does not sanitize or encode the output, so the script runs immediately in the browser.

****Impact:****

An attacker can execute arbitrary scripts in users' browsers, potentially stealing cookies, session information, or defacing pages.

****Fix:****

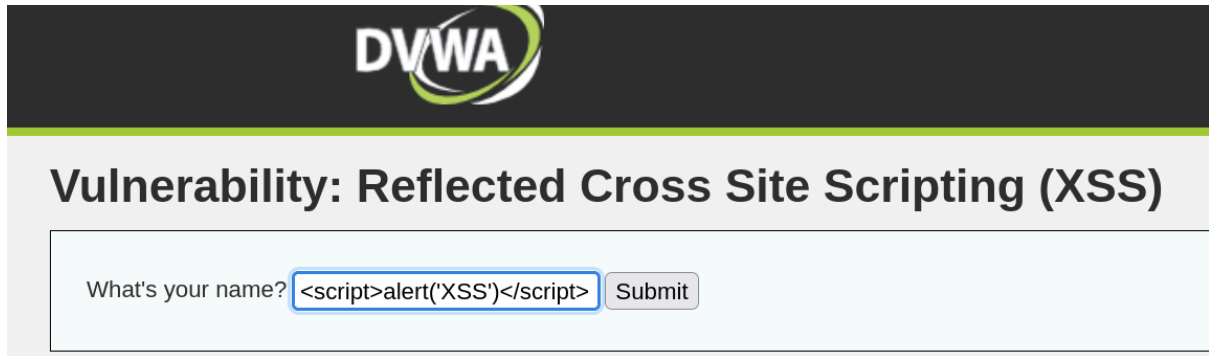
Sanitize and escape user input before rendering it in the page, implement Content Security Policy (CSP), and validate input to allow only expected characters.

Test 2 — Medium Security Level

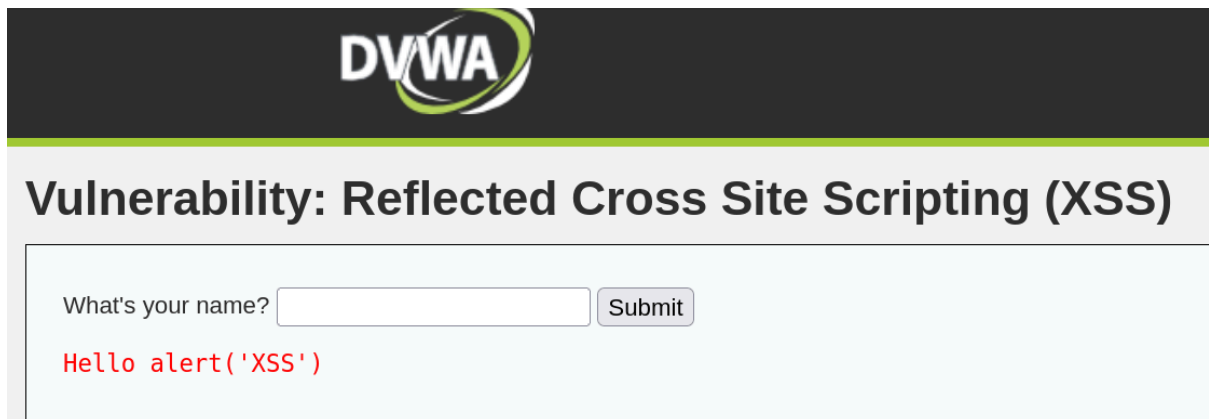
****Input:****

```
<script>alert('XSS')</script>
```

****Screenshot (Before):****

A screenshot of the DVWA (Damn Vulnerable Web Application) interface. At the top is a dark header with the DVWA logo. Below it is a light gray banner with the text "Vulnerability: Reflected Cross Site Scripting (XSS)". The main content area is white and contains a form with the label "What's your name?". The input field contains the payload "<script>alert('XSS')</script>". To the right of the input field is a "Submit" button.

****Screenshot (After):****

A screenshot of the DVWA interface after the payload was submitted. The header and banner are the same. The input field is now empty. Below the input field, the text "Hello alert('XSS')" is displayed in red, indicating that the alert function was successfully executed.

****Explanation:****

Medium security applies input filtering or encoding. Certain payloads may be blocked, but XSS is still conceptually possible.

****Impact:****

The risk of script execution is reduced but not eliminated. In real-world applications, clever bypasses could still allow attacks.

****Fix:****

Sanitize and encode output, apply strict input validation, and use CSP headers to limit script execution.

